Střední průmyslová škola Třebíč

Maturitní práce

SPOLEČENSKÁ HRA PEXESO

Profilová část maturitní zkoušky

Studijní obor: Informační technologie

Třída: ITA4

Školní rok: 2025/2026 Tomáš Karmazin

Zadání práce

Zadání ročníkové práce

Obor studia: **18-20-M/01 Informační technologie**

Jméno a příjmení:	**Tomáš Karmazin**		
Třída:	**ITA4**	Školní rok:	**2025/2026**
Číslo tématu:	**33**		
Název tématu:	**Společenská hra PEXESO**		
Rozsah práce:	**15 - 25 stránek textu**		

Specifické úkoly, které tato práce řeší:

Hra Pexeso je hra pro 2 až 6 hráčů nebo proti počítači. Počítač náhodně rozmístí obrázkové karty rubovou stranou navrch. Hráč otočí libovolné tři karty lícem navrch. Jestliže jsou karty shodné, získává bod a může pokračovat ve hře otočením další trojice karet. Jestliže jsou karty rozdílné, otočí se rubem navrch a ve hře pokračuje další hráč. Program bude obsahovat nabídku – nová hra, otevření, uložení, skóre, nastavení. Nová hra požaduje jména hráčů. Uložení a otevření rozehrané hry. Skóre obsahuje tabulku jméno hráče, výhra/prohra, počet nasbíraných karet, počet všech karet, kde půjde vyhledávat a filtrovat podle všech položek. Nastavení, zahrnuje zapnutí/vypnutí zvuku, počet hráčů, počet karet, druh karet, obtížnosti počítače. Obrázky na kartách budou vlastním dílem autora a půjde vybírat z několik sad. Vývojové prostředí VS, jazyk C#, sdílení a ukládání GitHub.

Termín odevzdání:	**29. března 2026, 23.00**
Vedoucí projektu:	**Mgr. Andrea Odehnalová**
Oponent:	**Ing. Marek Hrozníček**
Schválil:	**Ing. Petra Hrbáčková, ředitelka školy**

ABSTRAKT

Práce se věnuje vývoji desktopové aplikace v jazyce C# (.NET), která modifikuje hru pexeso pro vyhledávání trojic karet a převádí ji do digitální podoby. Implementace zahrnuje událostmi řízené programování, optimalizaci grafiky pomocí Sprite Atlasu a perzistentní ukládání dat přes binární serializaci. Klíčovým přínosem je algoritmus AI, který pomocí pravděpodobnostní matice simuluje kognitivní limity lidského hráče v několika úrovních obtížnosti.

KLÍČOVÁ SLOVA

pexeso, C#, Windows Forms, událostmi řízené programování, Sprite Atlas, umělá inteligence, Filestream, Vlastnost Tag, binární serializace

ABSTRACT

This thesis focuses on the design and development of a desktop application in C# (.NET), which digitizes and modifies the traditional memory game concept by implementing a triple-card matching mechanic. The technical implementation utilizes event-driven programming, graphical optimization via Sprite Sheets (Sprite Atlas), and persistent data storage using binary serialization. A key contribution of the project is a custom AI algorithm that employs a probability matrix to simulate the cognitive limits of a human player across multiple difficulty levels.

KEYWORDS

Memory Game, C#, Windows Forms, Event-driven programming, Sprite Atlas, Artificial Intelligence, FileStream, Tag Property, Binary Serialization

PODĚKOVÁNÍ

Děkuji Mgr. Andrei Odehnalové za cenné připomínky a rady, které mi poskytla při vypracování maturitní práce.

V Třebíči dne 29. března 2026 podpis autora

PROHLÁŠENÍ

Prohlašuji, že jsem tuto práci vypracoval/a samostatně a uvedl/a v ní všechny prameny, literaturu a ostatní zdroje, které jsem použil/a.

V Třebíči dne 29. března 2026

Tomáš Karmazin

Obsah

Úvod ... **7**
1 Teoretická část ... **8**
1.1 Pexeso a převedení procesu a pravidel hry do digitální verze ... **8**
1.1.1 Pravidla hry Pexeso ... 8
1.1.2 Převod pravidel do digitální verze ... 8
1.2 Zvolené technologie a platforma ... **9**
1.2.1 Platforma .NET Framework a Windows Forms ... 9
1.2.2 Systém formulářů ... 9
1.3 Uživatelské rozhraní a hardwarová akcelerace ... **10**
1.3.1 Plynulé Uživatelské rozhraní ... 10
1.3.2 Načítání velkého množství kartiček ... 10
1.3.3 Hardwarová akcelerace a řešení problému s problikáváním ... 11
1.4 Datové struktury a systém souborů ... **11**
1.4.1 Vhodné datové struktury ... 11
1.4.2 Binární soubory pro ukládání dat a jejich výhody ... 12
1.5 Multimédia a zvukový systém ... **13**
1.5.1 Zvukový systém a třída SoundPlayer ... 13
2 Praktická část ... **14**
2.1 Uživatelské prostředí ... **14**
2.1.1 Práce s jedním hlavním formulářem ... 14
2.1.2 Zvukový systém ... 16
2.1.3 Tmavý a bílý režim ... 16
2.2 Herní mechaniky ... **17**
2.2.1 Dynamické generování hry ... 17
2.2.2 Systém identifikátorů ... 18
2.2.3 Získávání obrázku na přední straně kartičky ... 18
2.2.4 Odkrytí kartiček ... 19
2.3 Umělý oponent ... **20**
2.3.1 Třída AI ... 20
2.3.2 Paměť ... 20
2.3.3 Obtížnosti ... 21

2.3.4 Rozhodovací strom ... 21
2.4 Ukládání uživatelských preferencí ... 22
2.4.1 Binární soubory DAT ... 22
2.4.2 Třída Filestream ... 22
2.4.3 Čtení struktur ... 22
2.5 Ukládání a načítání uložených her ... 23
2.5.1 Jak to funguje ... 23
2.5.2 Třída pro záznam ... 23
2.6 Systém skóre ... 25
2.6.1 Soubor historie.dat ... 25
2.6.2 Zpracování systému ... 25
2.7 Optimalizace ... 26
2.7.1 Zpracování hardwarové akcelerace a vyřešení problémů s blikáním ... 26
Závěr ... 28
Seznam použitých zdrojů ... 30
Seznam použitých symbolů a zkratek ... 32
Seznam obrázků ... 33

Úvod

Převod klasických stolních her do digitální podoby představuje ideální příležitost pro komplexní procvičení návrhu softwaru, algoritmizace a tvorby uživatelského rozhraní. Tato ročníková práce se věnuje vývoji desktopové počítačové verze hry Pexeso. Hlavní motivací pro výběr tohoto konkrétního tématu byla snaha vytvořit nejen plně funkční a interaktivní aplikaci, ale především uplatnit a prohloubit teoretické znalosti objektově orientovaného programování (OOP), správy paměti a perzistence dat v reálném softwarovém projektu.

Téma je koncipováno tak, aby přesáhlo rámec původní hry. Tradiční formát je zde inovován implementací mechanismu pro hledání trojic shodných karet namísto klasických dvojic. Hlavním cílem práce je tak navrhnout a naimplementovat robustní herní systém, který podporuje hru více hráčů v rozmezí 2 až 6 hráčů, nebo režim jednoho hráče proti umělé inteligenci. Mezi klíčové dílčí cíle patří vývoj dynamicky se přizpůsobujícího uživatelského rozhraní podle zvoleného počtu karet, vytvoření spolehlivého systému pro ukládání a načítání rozehraných partií a implementace statistik, které dlouhodobě zaznamenávají úspěšnost jednotlivých hráčů.

Veškerý vývoj a verzování zdrojových kódů probíhá prostřednictvím systému Git a je zálohován na platformě GitHub.

Práce je logicky strukturována do několika stěžejních kapitol. Po tomto úvodu následuje část věnovaná návrhu uživatelského rozhraní a dynamickému generování herní mřížky. Následující kapitoly detailně rozebírají vnitřní architekturu aplikace, zpracování herní logiky, chování umělé inteligence a technické specifikace práce s binárními soubory.

1 Teoretická část

Tato kapitola popisuje teoretický návrh a postupy pro převedení klasické stolní hry do digitální podoby.

1.1 Pexeso a převedení procesu a pravidel hry do digitální verze

Tato podkapitola se zaměřuje na pravidla pexesa a jejich převod do digitální podoby.

1.1.1 Pravidla hry Pexeso

Pexeso je klasická stolní karetní hra, jejímž primárním účelem je procvičení paměti a soustředění. Hra se standardně hraje s balíčkem karet, který obsahuje sudý počet herních prvků, přičemž každá karta má v balíčku přesně jeden vizuálně identický pár. V této digitální verzi jsou však místo tradičních dvojic karet použity trojice. Na počátku hry jsou všechny tyto karty náhodně rozloženy na herní plochu přední stranou dolů. [1]

Hráči se pravidelně střídají v tazích. Během jednoho tahu má hráč za úkol otočit přesně určený počet karet přední stranou nahoru. Pokud jsou odhalené obrázky na všech otočených kartách shodné, hráč tyto karty získá, připíše si příslušný počet bodů a má právo pokračovat dalším tahem. V případě, že se obrázky neshodují, karty se otočí zpět přední stranou dolů a na řadu přichází další hráč. Vítězem celé partie se stává ten, kdo po úspěšném rozebrání všech karet z herní desky dosáhne nejvyššího celkového skóre. [1]

1.1.2 Převod pravidel do digitální verze

Digitalizace fyzické karetní hry představuje komplexní úkol mapování reálných herních mechanik na abstraktní softwarové datové struktury a algoritmy. Zatímco ve fyzickém světě se o dodržování pravidel a sledování stavu hry starají lidští aktéři, v digitálním pojetí musí roli nestranného rozhodčího převzít herní smyčka aplikace.

Základem tohoto převodu je takzvaný stavový automat. Každá herní karta zde nevystupuje jako fyzický objekt, ale jako instance třídy či datový uzel nesoucí klíčové informace: stav viditelnosti (skrytá, otočená, nebo vyřazená) a především unikátní identifikátor. Pro tento účel jsem zvolil systémovou vlastnost Tag, do které lze ukládat

libovolný prvek (datového typu object), což umožňuje ke každému ovládacímu prvku přímo přidružit identifikační číslo pro výřez obrázku a příslušnost k logické trojici. Mechanismus tahu je pak realizován skrze zachytávání uživatelských událostí, konkrétně kliknutí myší. Program musí v reálném čase validovat počet aktuálně otočených karet a po dosažení stanoveného limitu dočasně zablokovat interaktivitu ostatních prvků, aby nedošlo k narušení herní sekvence. Součástí digitalizace je i eliminace nežádoucího chování. Aplikace například musí striktně ignorovat vstupy uživatele ve chvíli, kdy není na řadě. [2]

1.2 Zvolené technologie a platforma

Tato část práce zdůvodňuje výběr technologického stacku, který tvoří základ celého projektu.

1.2.1 Platforma .NET Framework a Windows Forms

Pro implementaci aplikace jsem zvolil platformu .NET Framework ve spojení s technologií Windows Forms. Platforma .NET Framework od společnosti Microsoft tvoří ucelený ekosystém poskytující kvalitní vývojové prostředí a velmi obsáhlou standardní knihovnu tříd. Tyto komponenty vývojářům výrazně usnadňují tvorbu programů. [3]

Technologie Windows Forms představuje aplikační rozhraní zaměřené na tvorbu grafických desktopových aplikací (GUI) pro operační systémy rodiny Windows. Vývoj aplikace probíhá pomocí událostmi řízeného programování (event-driven programming). Grafické prvky generují události (events), na které aplikace reaguje voláním specifických bloků kódu (event handlerů). To vývojáři umožňuje jednoduše svázat například stisk konkrétního vizuálního tlačítka s funkcí, která otočí herní kartu. [3]

1.2.2 Systém formulářů

Při vývoji aplikací v prostředí Windows Forms je hlavním stavebním prvkem třída Form. Formulář představuje samotné okno na obrazovce počítače, které slouží jako vizuální a logický kontejner pro všechny ostatní komponenty (tlačítka, textová pole, časovače a další ovládací prvky). [4]

V architektonicky ucelené hře je zásadní správně navrhnout distribuci formulářů. Ideálním postupem je zachování jednoho hlavního formuláře, ve kterém se otevírají další formuláře. Tímto způsobem nedochází k neustálému rušivému přepínání mezi jednotlivými okny a používání programu působí plynulejším dojmem.

1.3 Uživatelské rozhraní a hardwarová akcelerace

Tato část práce se věnuje návrhu a technické realizaci uživatelského rozhraní (UI), které tvoří komunikační most mezi hráčem a logikou aplikace.

1.3.1 Plynulé Uživatelské rozhraní

Hlavním požadavkem na moderní softwarový produkt je responzivní a plynulé uživatelské rozhraní. Ve chvíli, kdy aplikace spravuje několik desítek prvků, může docházet k degradaci grafického výkonu. Základní překreslovací logika Windows Forms při změně na formuláři postupuje sekvenčně: nejprve vymaže danou oblast barvou pozadí a následně na ni postupně vykresluje jednotlivé prvky. Vzhledem k obnovovací frekvenci monitorů tento dvoufázový postup často odhaluje okem zachytitelné prodlevy mezi vymazáním a opětovným vykreslením elementů, což způsobuje nepříjemný vizuální defekt označovaný jako problikávání (flickering). Plynulé UI znamená dosažení stavu, kdy grafické iterace probíhají bez rušivých artefaktů a operace na pozadí neblokují hlavní vlákno, čímž se předchází zamrzání samotného programu. [5]

1.3.2 Načítání velkého množství kartiček

V případě aplikace, která musí dynamicky vygenerovat herní plochu s několika desítkami prvků, představuje správa grafických zdrojů kritický bod pro výkon a paměťovou náročnost. Přístup spočívající v odděleném načítání desítek samostatných obrázkových souborů z pevného disku pro každou jednotlivou herní kartu generuje vysokou zátěž na vstupně-výstupní (I/O) operace operačního systému. Každý soubor s sebou navíc nese vlastní hlavičku a paměťovou režii, což při větším měřítku vede k fragmentaci paměti a zpomalení startu hry. [6]

Z toho důvodu se v softwarovém vývoji využívá technika zvaná grafický atlas (Texture Atlas či Sprite Sheet). Veškeré textury karet jsou předem spojeny do jediného

rozsáhlého obrazového souboru. Aplikace tak při spuštění přistupuje k pevnému disku pouze jednou a celý atlas načte do operační paměti jako jeden souvislý blok dat. [7]

1.3.3 Hardwarová akcelerace a řešení problému s problikáváním

Pro potlačení problikávání je nezbytné sáhnout k optimalizačním technikám, z nichž nejzákladnější je takzvaný double buffering. Princip double bufferingu spočívá ve změně paradigmatu vykreslování – místo okamžitého vykreslování jednotlivých grafických změn přímo na monitor (do zobrazovací paměti) se nejprve vytvoří virtuální bitmapa (off-screen buffer) uložená ve standardní operační paměti. Veškeré příkazy ke kreslení se realizují v této skryté paměti. Teprve po dokončení kompletní grafické scény je tento obraz jednorázově zkopírován na obrazovku. K probliknutí tak z principu nemůže dojít, neboť uživatel nevidí prázdné mezikroky. [8]

V jednodušších aplikacích postačí v prostředí .NET nastavit vlastnost DoubleBuffered daného prvku na hodnotu true. U složitých hierarchických architektur (např. panel s mnoha kartami uvnitř formuláře s obrázkem na pozadí) však standardní mechanismy selhávají. V takové situaci je potřeba využít hlubších zásahů na úrovni parametrů okna. Úpravou vlastnosti CreateParams formuláře o příznak zvaný WS_EX_COMPOSITED (hexadecimální kód 0x02000000) se aktivuje akcelerovaná kompozice okna. Systém Windows v tomto režimu přesune práci na úroveň správce oken (Desktop Window Manager), který vynutí kompozitní překreslení formuláře a všech jeho potomků takzvaně zespoda nahoru ve sdíleném vyrovnávacím prostoru. Výsledkem je stabilní obraz bez jakýchkoli grafických anomálií a blikání i při náročných vizuálních transformacích. [9]

1.4 Datové struktury a systém souborů

Tato část práce se zaměřuje na vnitřní organizaci dat a způsob jejich trvalého uchovávání v hierarchii operačního systému.

1.4.1 Vhodné datové struktury

Aplikace potřebuje bezpečně, logicky a efektivně uchovávat velké množství dat, což je nezbytné pro její správný chod. Volba vhodné datové struktury je v objektovém programování klíčová pro udržení celkové výkonnosti. Platforma .NET nabízí širokou škálu řešení, od základních polí (arrays) po generické kolekce. Klasická

jednorozměrná i vícerozměrná pole představují souvislý blok paměti o přesně dané velikosti. Tato vlastnost zajišťuje velmi rychlý přístup k prvku na základě jeho číselného indexu s minimální výpočetní a paměťovou režií. Pole jsou proto ideální pro statická data. [10]

Naopak dynamické elementy hry, jejichž celkový počet se za běhu aplikace výrazně proměňuje (jako například seznam karet aktuálně vybraných během probíhajícího tahu, seznam nalezených souborů z dřívějších uložení nebo samotná databáze tabulek skóre), vyžadují flexibilitu. Pro tyto účely se využívá generická kolekce List<T>. Vnitřní mechanika třídy List je vystavěna nad standardním polem, avšak při vyčerpání kapacity program na pozadí automaticky alokuje pole o větší velikosti a data do něj zkopíruje. To výrazně usnadňuje práci s dynamicky se měnícím množstvím dat. [11]

1.4.2 Binární soubory pro ukládání dat a jejich výhody

Perzistentní data, která mají aplikaci přežít i po jejím vypnutí, jako je uživatelské nastavení, historie všech dohraných partií či uložení stavu pro opětovné nahrání musejí být zapsána na pevný disk počítače. Ze široké palety formátů jsem si v této práci zvolil formát binárního souboru s koncovkou .dat.

Výhody binárního zápisu oproti běžnému textovému ukládání spočívají primárně v rychlosti a efektivitě. V textovém souboru se každý znak či číslice konvertuje a ukládá samostatně. Například rozsáhlé číselné skóre uložené v textové podobě zabere tolik bajtů, kolik má znaků, a při opětovném čtení musí dojít k algoritmickému převodu textu zpět na numerickou hodnotu. Binární přístup, realizovaný v jazyce C# skrze provázání třídy FileStream s obalovými třídami BinaryWriter a BinaryReader, naopak vezme blok dat tak, jak je uložen v operační paměti, a tento kompaktní binární řetězec zkopíruje rovnou na disk. Tento postup výrazně šetří výpočetní čas procesoru při načítání historie a minimalizuje nároky na diskový prostor. Dalším benefitem je, že surový soubor s příponou .dat nelze snadno přečíst v běžném textovém editoru. Uživatel by pro úspěšné čtení musel znát přesnou strukturu zapsaných dat, která mu není z obsahu souboru přímo patrná. [12]

1.5 Multimédia a zvukový systém

Tato podkapitola se věnuje integraci a správě multimediálních prvků, které zvyšují interaktivitu a uživatelský komfort aplikace.

1.5.1 Zvukový systém a třída SoundPlayer

Interaktivita aplikací nebývá reprezentována pouze vizuální odezvou, ale pro obohacení dojmů hráče je velmi často doprovázena zvukovými efekty. Tato auditivní zpětná vazba může představovat například zvuk kliknutí v menu nebo zvukovou signalizaci selhání při hledání shodných karet. V rámci ekosystému Windows Forms poskytuje .NET Framework přímočaré řešení ve formě vestavěné třídy SoundPlayer, která je obsažena ve jmenném prostoru System.Media. Vyznačuje se přímou vazbou na formát nekomprimovaných zvukových souborů .wav. [13]

Klíčovou implementační zásadou v interaktivním prostředí je nezbytné využití asynchronního přehrávání. Třída disponuje metodou Play(), která zajistí vytvoření nového pracovního vlákna operačního systému výhradně pro obsluhu přehrávaného zvuku. Tím se zaručuje, že hlavní vlákno aplikace (odpovědné za zpracování událostí) zůstane responzivní a nedojde k nežádoucímu „zamrznutí“ okna po dobu přehrávání zvukové stopy. To je pro zachování plynulého uživatelského rozhraní (UI), zmiňovaného v předchozích bodech, naprosto kritické. [13]

2 Praktická část

Tato kapitola se zaměřuje na konkrétní implementaci navrhovaných postupů.

2.1 Uživatelské prostředí

Uživatelské rozhraní je důležitá část každého programu. Má zásadní roli na přijetí programu uživateli.

2.1.1 Práce s jedním hlavním formulářem

Při návrhu struktury aplikace bylo klíčovým rozhodnutím vyhnout se neustálému otevírání a zavírání nových nezávislých oken operačního systému Windows, což by mohlo působit nesourodě a rušivě. Tento problém byl vyřešen centralizací navigace do jednoho nadřazeného okna reprezentovaného třídou PEXESO v souboru Main.cs. Toto hlavní okno slouží jako kontejner, do kterého se dynamicky načítají veškeré ostatní herní obrazovky. K tomu slouží klíčová metoda OtevreniFormu, která jako svůj vstupní parametr přijímá instanci nového formuláře, který se má uživateli zobrazit. Metoda nejprve záměrně pozastaví překreslování hlavního okna pomocí metody SuspendLayout, následně uzavře a uvolní z paměti aktuálně aktivní formulář (pokud nějaký existuje) a nově příchozímu formuláři odstraní horní systémové ohraničení (vlastnost FormBorderStyle = FormBorderStyle.None). Dále přenastaví jeho chování tak, aby s ním aplikace zacházela jako s běžným ovládacím prvkem, a nikoliv jako se samostatným oknem (TopLevel = false). Tento vnitřní formulář se poté roztáhne přes celou dostupnou plochu pomocí vlastnosti DockStyle.Fill a automaticky se na něj aplikuje aktuálně zvolený barevný režim. Na samém konci metoda přidá formulář do kolekce Controls a obnoví překreslování.

Obrázek 2.1 Hlavní menu

2.1.2 Zvukový systém

Pro obohacení uživatelského zážitku je aplikace vybavena responzivním zvukovým systémem. Samotná logika přehrávání zvukových notifikací je opět spravována zcela centrálně z hlavního okna aplikace, konkrétně v metodě prehratZvuk. Centralizace přehrávání na jedno místo je výhodná, jelikož umožňuje okamžitě reagovat na preference uživatele napříč celou aplikací – pokud je v nastavení zvuk vypnutý, metoda po svém zavolání pouze vyhodnotí stav proměnné a rovnou se ukončí, aniž by se zbytečně alokovaly systémové prostředky. Pokud jsou zvuky povoleny, metoda vybírá konkrétní stopu na základě přijatého parametru (celočíselného indexu). Například je-li požadován zvuk s indexem 0, aplikace pomocí vestavěné třídy SoundPlayer načte do paměti z prostředků aplikace (Properties.Resources) zvukový soubor představující běžné kliknutí. Následně se na instanci přehrávače zavolá metoda Play(), která zvuk přehraje asynchronně v samostatném pracovním vlákně. To zaručuje, že nedojde k nechtěnému zamrznutí uživatelského rozhraní v okamžiku kliknutí.

2.1.3 Tmavý a bílý režim

Moderní softwarové standardy často vyžadují poskytnutí možnosti volby vzhledu ke zmírnění únavy zraku. Z tohoto důvodu aplikace obsahuje funkci pro přepínání mezi tmavým a světlým režimem. Nastavení barevného profilu se načítá z konfiguračního souboru a je globálně uloženo v paměti okna. Změna celkového designu se plošně aplikuje pomocí speciálně navržené sady iterativních metod AplikujSvetlyRezim a AplikujTmavyRezim. Tyto metody přijímají jako parametr referenci na překreslovaný formulář a pomocí cyklu foreach postupně a systematicky projdou každý jednotlivý vizuální prvek (tzv. Control) obsažený uvnitř okna – včetně tlačítek, informačních popisků, složitějších panelů či roletových menu. U každého prvku se na základě jeho typu zanalyzuje požadavek a rekurzivně se změní jeho primární barva pozadí (BackColor), barva písma (ForeColor) a parametry plochého zobrazení, jako jsou tloušťka a barva ohraničení, přesně podle předem definovaného vzorníku palet Color.

2.2 Herní mechaniky

Tato podkapitola detailně rozebírá logiku modifikovaného pexesa. Dále jsou popsány algoritmy pro generování hracího pole a základní systémy, na kterých aplikace funguje.

2.2.1 Dynamické generování hry

Vlastní herní plocha není ve vizuálním návrháři fixně vytvořena předem. Celý její herní obsah a mřížka se dynamicky generují v reálném čase až při spuštění nové partie ve formuláři Game.cs. Celý proces je ovlivněn počátečními parametry načtenými z uživatelských preferencí – typicky celkovým počtem karet a volbou vizuálu. Tím je dosaženo vysoké míry flexibility. Herní logika nejprve matematicky vypočítá vhodné rozložení prvků. Samotná interaktivní tlačítka, která reprezentují jednotlivé herní karty, se instancují programově přímo ze zdrojového kódu pomocí cyklů. Každému

```
private void VytvorKartu(int i, int tagValue, int pocetSloupcu, bool jeAktivni)
{
    Button btn = new Button();
    btn.Name = "karta" + i;
    btn.Size = new Size(120, 120);
    btn.Left = (i % pocetSloupcu) * 125;
    btn.Top = (i / pocetSloupcu) * 125;
    btn.Tag = tagValue;
    btn.BackColor = barvaTlacitek;
    btn.FlatAppearance.BorderSize = 3;
    btn.FlatAppearance.BorderColor = barvaTextu;
    btn.BackgroundImageLayout = ImageLayout.Stretch;
    btn.Font = new Font("Roboto", 10, FontStyle.Bold);
    btn.ForeColor = barvaTextu;

    if (jeAktivni)
    {
        btn.Text = (i + 1).ToString();
    }
    else
    {
        btn.Text = "";
    }

    btn.Click += Karta_Click;

    herniPoleProSave.Add(btn);

    if (jeAktivni)
    {
        panel1.Controls.Add(btn);
```

Obrázek 2.2 Metoda pro vytvoření tlačítka reprezentujícího kartičku

z nich se ihned při vytvoření přiřadí rozměry a souřadnice, nastaví se vzhled odpovídající vybranému barevnému režimu a především se na něj naváže obsluha události stisku (Click), která následně předá řízení vyhodnocovacímu algoritmu.

2.2.2 Systém identifikátorů

Rozpoznání shodných karet představuje nejdůležitější mechaniku pexesa. Program proto potřebuje spolehlivý způsob k identifikaci obsahu karty, aniž by musel složitě analyzovat samotný obrazový soubor. Každé kartě je tak na pozadí přiřazen jednoznačný identifikátor (ve formě čísla), který je uložen do vlastnosti Tag. Tato vlastnost slouží ke spárování – všechny herní prvky tvořící požadovanou trojici mají ve vlastnosti Tag uloženo totožné číslo. Ve chvíli, kdy hráč vybere karty a počet označených prvků dosáhne maximálního limitu pro tah, hra zablokuje další uživatelské interakce a předá vybrané komponenty do dočasného seznamu ke kontrole. Během procesu vyhodnocování se extrahují hodnoty z vlastnosti Tag a navzájem se porovnají. Pokud je nalezena shoda u všech zvolených objektů, jedná se o validní tah a hra pokračuje inkrementací skóre a odstraněním karet z herní plochy. V opačném případě se spustí časovač (Timer), který poskytne hráči krátký časový prostor pro zapamatování špatného výběru. Po uplynutí nastavené doby se karty otočí zpět do výchozí polohy.

2.2.3 Získávání obrázku na přední straně kartičky

Pro snížení výpočetní a paměťové náročnosti a zajištění snadné modifikace vizuální stránky hry aplikace nenačítá desítky samostatných obrazových souborů pro přední strany karet, nýbrž využívá techniku takzvaného grafického atlasu (Sprite Sheet). Každý tento soubor má rozměry 256 × 5120 pixelů, což znamená, že jedna sada obsahuje 20 unikátních obrázků kartiček. V momentě, kdy je po otočení karty nezbytné vykreslit její přední stranu, aplikace za pomoci grafických knihoven a na základě hodnoty vlastnosti Tag (která slouží jako unikátní identifikátor) matematicky vypočítá pozici konkrétního motivu v atlasu. Následně se tato přesná část zdrojového obrázku vyřízne a vykreslí na povrch dané karty.

```
Počet odkazů: 4
private Image kartaZistkatObrazek(int index)
{
    Bitmap bmp = new Bitmap(120, 120);
    using (Graphics g = Graphics.FromImage(bmp))
    {
        g.InterpolationMode = InterpolationMode.NearestNeighbor;
        g.PixelOffsetMode = PixelOffsetMode.Half;
        g.Clear(Color.Transparent);
        int srcX = index * 256;
        Rectangle zdroj = new Rectangle(srcX, 0, 256, 256);
        Rectangle cil = new Rectangle(0, 0, 120, 120);
        g.DrawImage(atlasObrazku, cil, zdroj, GraphicsUnit.Pixel);
    }
    return bmp;
}
```

Obrázek 2.3 Metoda pro získání výřezu kartičky

2.2.4 Odkrytí kartiček

Při testování chodu hry bylo potřeba vytvořit mechanismus pro rychlé zobrazení výřezů (předních stran) na všech kartách současně. Pro tento účel byla do kódu vložena speciální diagnostická metoda. Tato metoda naslouchá globálním událostem klávesnice díky aktivované vlastnosti formuláře KeyPreview. V momentě, kdy je detekován stisk klávesy, konkrétně * (hvězdička), vyvolá se metoda OdkrytKarty. Aplikace bez ohledu na aktuální stav herního pořadí v krátkém časovém intervalu iteruje skrze všechny herní prvky na ploše a dočasně odkryje jejich přední strany. Tímto způsobem může vývojář jediným stiskem klávesy vizualizovat kompletní rozložení hrací plochy, aniž by došlo k narušení vnitřní logiky nebo stavového automatu hry.

2.3 Umělý oponent

Tato kapitola se zaměřuje na jeden z nejlepších aspektů softwarové verze této hry, umělého oponenta. Věc, která ve fyzické verzi není možná. Je zde vysvětleno fungování paměti umělého oponenta a veškeré metody, které dohromady úspěšně napodobují reálného hráče.

2.3.1 Třída AI

Pro zajištění podpory režimu pro jednoho hráče byl zkonstruován samostatný virtuální oponent, který je logicky oddělen od struktury uživatelského rozhraní do samostatné třídy umělé inteligence v souboru AI.cs. V rámci čistoty návrhu tato třída přímo nemanipuluje s vizuálními prvky; pracuje výhradně s abstraktními daty (identifikátory a souřadnicemi) a skrze veřejné metody navrací své instrukce. Ty jsou následně v hlavním okně interpretovány a převedeny na simulované kliknutí na herní ploše.

2.3.2 Paměť

Klíčovým aspektem jakékoli umělé inteligence hrající pexeso je schopnost udržet si přehled o herním poli. K tomuto účelu slouží vyhrazené dvourozměrné pole, které funguje jako paměťový model oponenta. Souřadnice v tomto poli přesně definují, kde se konkrétní identifikátory karet nacházejí. Kdykoli je během hry karta odhalena a následně opět skryta, algoritmus tuto událost prostřednictvím příslušné metody zpracuje a uloží. První index zmíněné datové struktury představuje identifikátor (Tag) karty a druhý index zaznamenává pořadí nálezu v rámci trojice. To umělé inteligenci umožňuje efektivně vyhledávat a následně kompletovat shodné herní prvky.

Paměť umělé inteligence není statickým prvkem. Během dynamického průběhu partie z herního pole fyzicky odcházejí ulovené bloky karet na bodová konta živých či počítačových soupeřů. Pokud by se paměť stroje průběžně nečistila, odkazovala by struktura AI na grafické prvky, které již v aplikaci neexistují. Z toho důvodu je implementována čisticí metoda OdstranKartyZPameti. Ta je volána při každé nalezené shodě kteréhokoli hráče a bez ohledu na nastavenou náročnost smaže dotčené herní instance z ukazatelů paměťové matice AI. Tím udržuje systém za všech okolností plně konzistentní.

2.3.3 Obtížnosti

Algoritmus umělé inteligence byl navržen tak, aby jeho výkonnost mohla být v globálním nastavení modifikována parametrem obtížnosti. Změna náročnosti nespočívá v úpravě samotného rozhodovacího algoritmu, ale v zavedení prvku pravděpodobnosti při ukládání informací do paměti. Díky tomu oponent chybuje a působí přirozenějším, lidským dojmem.

Při nejnižší úrovni obtížnosti (0) je proces ukládání dat do paměti zcela vynechán. AI v tomto režimu funguje pouze jako generátor náhodných voleb, který ze zbývajících karet na ploše vybere tři náhodné instance. U vyšších úrovní (1 a 2) je definován procentuální koeficient úspěšnosti zapamatování (např. 20 % pro střední a 55 % pro vysokou obtížnost). Každý pokus o zápis odhalené karty do paměťového pole je vyhodnocen generátorem pseudonáhodných čísel. Pouze pokud výsledná hodnota odpovídá stanovené toleranci, dojde k uložení pozice karty. Tato data pak AI využívá k efektivnímu kompletování trojic v následujících tazích.

2.3.4 Rozhodovací strom

Algoritmus výběru karet počítačem ve svém tahu funguje dle sekvenčního postupu pravidel. Když se umělý oponent dostane na řadu při výběru trojice kartiček, jeho první prioritou je iterace vlastního dvourozměrného pole, ve kterém hledá, zda v něm již nemá uloženou kompletní trojici s identickým parametrem Tag. Pakliže má štěstí a kompletní shoda v paměti existuje, logicky zařadí tyto známé objekty do fronty svých tahů a má vítězství v kapse. V situaci, kdy tato jistota chybí, se chování umělé inteligence přepne na proces objevování – provede vysoce rizikový, čistě náhodný tah z doposud neviděných karet. Bezprostředně poté provede křížovou kontrolu; zjistí, zda se vlastnost této nově objevené karty (která mu byla odtajněna) náhodou neshoduje s nějakou, kterou má již z minulosti zapsanou v paměti od jiných hráčů. V případě úspěchu si z paměti k této kartě okamžitě cíleně dolosuje patřičný zbytek.

2.4 Ukládání uživatelských preferencí

Tato část práce se zabývá implementací perzistentního ukládání uživatelských nastavení, která definují chování a vzhled aplikace při opakovaném spuštění.

2.4.1 Binární soubory DAT

Veškerá perzistentní data, která mají být trvale k dispozici a musí zůstat zachována i po ukončení programu, jsou zapisována na pevný disk počítače do konfiguračního souboru v systémové složce aplikace. Pro tento účel bylo využito binárního zápisu. Použití surového binárního formátu přináší značné výhody pro celkový výkon programu – proměnné (typicky číselné definice, jako je počet hráčů) jsou přenášeny přímo ve svém neparsovaném bajtovém rozložení z operační paměti do paměti diskové. Tím se redukuje celková velikost souboru na minimum a zároveň se běžným uživatelům znemožňuje obsah souboru upravovat pomocí textových editorů bez použití specializovaných binárních nástrojů.

2.4.2 Třída Filestream

Hlavní branou k perzistenci dat představuje v prostředí jazyka C# obsluha systémové třídy FileStream, kterou program hojně zapojuje prakticky v každém načítacím či zapisovacím okně, jako je tomu na počátku spuštění uvnitř Main.cs a Settings.cs. Souborový proud vyžaduje striktní znalost cesty k umístění souboru na lokálním disku a definování módu přístupu k souboru (např. FileMode.Create pro nové překreslení a zápis nastavení, či FileMode.Open k jeho pouhému vynesení z disku do paměti aplikace).

2.4.3 Čtení struktur

Datový proud samotný nemá tušení, k jakým proměnným čtené byty patří. K organizaci dat nad ním tak operují objektové třídy BinaryReader pro načítání do hry a BinaryWriter pro fyzický zápis na médium. Samotný algoritmus je sestaven tak, aby vždy přečetl správnou proměnnou a došlo tak k úspěšnému přečtení dat.

2.5 Ukládání a načítání uložených her

Tato podkapitola se věnuje mechanismům, které zajišťují kontinuitu herního zážitku prostřednictvím ukládání a opětovného načítání rozehraných partií.

2.5.1 Jak to funguje

Zásadním kamenem uživatelské přívětivosti v tomto projektu je mechanismus zprostředkovávající pozastavení aktuálního sezení pomocí tvorby ukládacích pozic. Tento koncept řeší problematiku přerušované digitální hry tak, že zaručuje hráči možnost uchovat kompletní a neměnný postup herní partie.

2.5.2 Třída pro záznam

Při inicializaci ukládání hra extrahuje veškerý svůj aktuální kontext a obalová třída za pomoci instance BinaryWriter začne vytvářet kompaktní blok záznamu do jednoho ze tří vybraných souborů. Klíčovým prvkem je uložení metadat, mezi která patří název uložené hry pro vizualizaci v nabídce načítání, pořadový index hráče (uživatele či AI), který má být po načtení na tahu, a celkový počet karet v herní konfiguraci. Součástí profilu relace je také kompletní výčet identifikátorů všech zúčastněných hráčů a jejich aktuální bodový stav.

Po metadatech následuje iterace samotného maticového pole herní plochy. Cyklus prochází každý herní prvek a zapisuje na disk jeho unikátní ID (vlastnost Tag) spolu s informací o jeho stavu – tedy zda se karta stále nachází na stole přední stranou dolů, nebo zda již byla nalezena a odstraněna ze scény. Modul LoadGame.cs následně zajišťuje vizualizaci všech dostupných uložených pozic a předává parametry do konstruktorů hlavní herní třídy, která na jejich základě dřívější rozložení mapy znovu sestaví.

Obrázek 2.4 Rozhraní nastavení

2.6 Systém skóre

Tato část práce se zaměřuje na vývoj a implementaci robustního systému pro evidenci herních výsledků a jejich následnou analýzu.

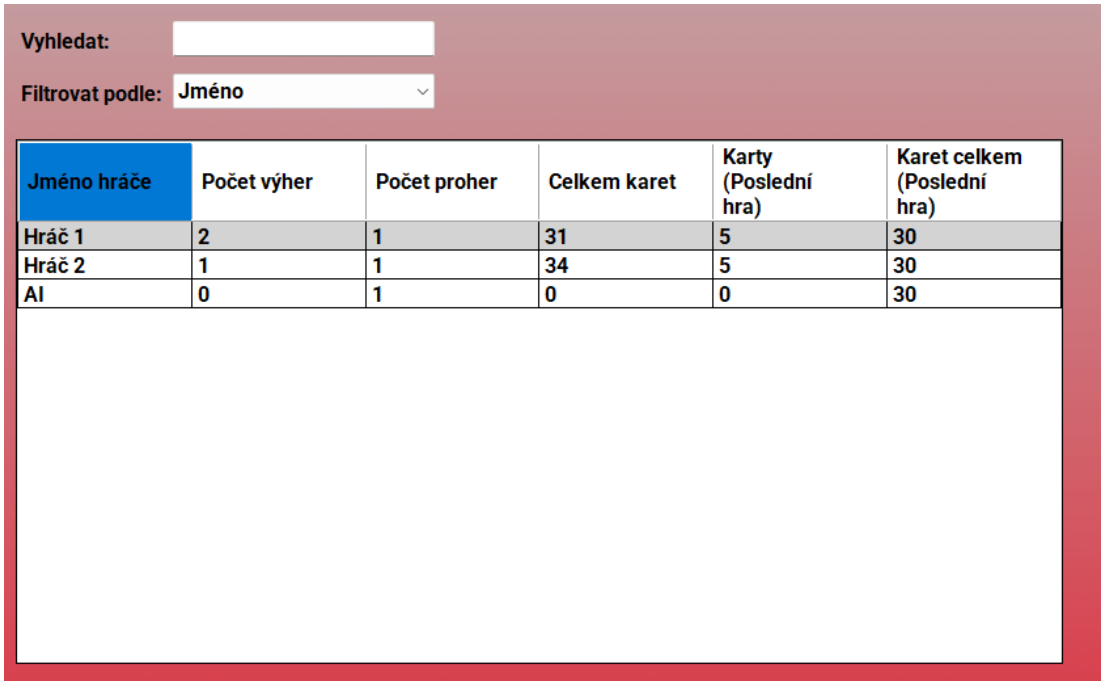
2.6.1 Soubor historie.dat

Trvalá tabulka rekordů tvoří základ dlouhodobé zpětné vazby, která v interaktivním prostředí buduje kompetitivní herní napětí. Veškeré individuální výsledky se shromažďují do nezávislého souboru trvalého bodového hodnocení – konkrétně do souboru history.dat. Tento archiv slouží jako historický přehled úspěšnosti hráčů napříč všemi herními seancemi.

2.6.2 Zpracování systému

Logická analýza a zpracování výsledků probíhá v momentě zobrazení závěrečné obrazovky s vyhodnocením vítězů. Algoritmus aplikace nejprve do operační paměti načte z binárního souboru rozsáhlé pole všech historicky registrovaných hráčů, včetně jejich statistik výher, proher a celkového počtu nasbíraných trojic. Následně se spustí proces aktualizace dat: program identifikuje vítěze i poražené aktuální partie a porovná jejich identity s databází historických profilů.

U nalezených shodných profilů program inkrementálně navýší příslušné hodnoty úspěšnosti. Pokud se jméno aktuálního uživatele v databázi doposud nevyskytovalo, vytvoří pro něj nový datový záznam. Poté jsou všechna aktualizovaná data zapouzdřena a celý datový soubor je hromadně přepsán na disk. Pro vizualizaci těchto dat v uživatelském rozhraní slouží formulář Score.cs. Ten načte historii z disku a s využitím dynamické vyhledávací vrstvy (kolekce List v kombinaci se strukturou ZaznamHrace) vloží výsledky do komponenty DataGridView. Tato tabulka je opatřena funkcí plynulé filtrace podle jména, počtu výher, proher a dalších parametrů.



Obrázek 2.5 Tabulka skóre

2.7 Optimalizace

Tato podkapitola rozebírá techniky využité k dosažení vysokého výkonu a nízké paměťové náročnosti aplikace, které je vhodná pro plynulost hry.

2.7.1 Zpracování hardwarové akcelerace a vyřešení problémů s blikáním

Nejvýraznějším grafickým nedostatkem standardní technologie Windows Forms je efekt tzv. problikávání (flickering). K tomuto jevu dochází především proto, že při objemnějších dynamických změnách (jako je masové překreslování pexeso karet) systém nejprve sekvenčně vymaže překreslovanou zónu barvou pozadí a teprve poté vykresluje jednotlivé objekty. Tento proces probíhá v několika krocích, což lidské oko vnímá jako nepříjemné blikání.

Aby projekt splnil kritéria vizuální plynulosti, byla nezbytná integrace softwarové a paměťové akcelerace. Ústředním řešením je modifikace pravidel přímo v jádrovém konstruktoru hlavní třídy aplikace (v modulu Main.cs). Prvním krokem je programové

vynucení dvojité vyrovnávací paměti nastavením vlastnosti DoubleBuffered na hodnotu true.

S ohledem na složitou strukturu podřízených prvků je tato optimalizace prohloubena systémovým nastavením pomocí metody SetStyle. Tímto aplikace předává vykreslovacímu jádru Windows specifické příznaky řízení (ControlStyles). Konkrétně parametry OptimizedDoubleBuffer v kombinaci s příznaky AllPaintingInWmPaint a UserPaint přebírají nízkoúrovňovou kontrolu nad vykreslováním a potlačují standardní systémové rutiny. Díky tomu je grafický cyklus sestaven primárně v oddělené vnitřní vyrovnávací paměti. Procesor pak na displej zkopíruje až kompletně sestavený pixelový výstup, čímž veškeré vizuální defekty a problikávání karet během jejich transformace zcela mizí.

Závěr

Cílem této práce bylo navrhnout a zrealizovat plně funkční desktopovou verzi klasické stolní hry Pexeso. Výsledkem této práce je funkční a stabilní softwarové řešení naprogramované v jazyce C# nad platformou .NET Framework s využitím technologie Windows Forms. Oproti tradiční stolní verzi hry nabízí tato digitální podoba inovativní a obtížnější mechaniku hledání trojic místo klasických párů, což před hráče staví vyšší nároky na paměť a soustředění, a zároveň poskytuje mnohem širší možnosti přizpůsobení.

Zásadním přínosem vytvořené aplikace je její flexibilita a důraz na uživatelský zážitek. Hra obsahuje komplexní systém nastavení, který umožňuje dynamicky upravovat parametry partie, jako je celkový počet karet na hrací ploše, vizuální motivy herních prvků či možnost plynulého přepínání mezi světlým a tmavým režimem uživatelského rozhraní (Světlý/Tmavý režim). Velká péče byla věnována samotnému běhu aplikace, kdy se díky hlubší integraci hardwarové akcelerace a dvojitého vyrovnávání (Double Buffering) podařilo zcela eliminovat nežádoucí problikávání grafických prvků. Výsledkem je čisté, responzivní a moderní rozhraní, které uživatele při hraní nijak neruší.

Aplikace rovněž úspěšně řeší absenci lidských protihráčů implementací umělé inteligence. Virtuální oponent byl navržen tak, aby dokázal simulovat lidské chování, včetně nedokonalé paměti. Díky volitelným úrovním obtížnosti, které pracují s pravděpodobností zapamatování odkrytých karet, je hra přístupná jak pro začátečníky, tak představuje výzvu pro pokročilé hráče.

Po technické stránce práce demonstruje pokročilé využití objektově orientovaného programování, dynamických datových struktur a efektivního zpracování dat. Zcela klíčovým prvkem z hlediska dlouhodobé hratelnosti je spolehlivý systém perzistence dat. Pomocí úsporného binárního zápisu byl vytvořen nejen mechanismus pro bezpečné ukládání uživatelských preferencí a stavu rozehraných her (tzv. Save/Load systém), ale také plně funkční databáze dlouhodobých výsledků. Tento systém udržuje historii výher, proher a nasbíraných bodů napříč všemi sezeními, čímž do hry vnáší silný kompetitivní náboj.

Celkově lze konstatovat, že stanovené cíle projektu byly beze zbytku naplněny. Vytvořený program není pouze prostou kopií fyzické hry, ale plnohodnotným interaktivním softwarem, který rozšiřuje původní koncept o nové mechaniky, trvalý postup a umělou inteligenci. Zvolená architektura navíc umožňuje případné budoucí rozšiřování, například o nové sady obrázků, síťovou hru pro více hráčů přes internet nebo přidání dodatečných vizuálních a zvukových efektů. Práce tak posloužila jako vynikající praktická ukázka prolnutí návrhu uživatelského rozhraní s komplexní herní logikou a správou dat.

Seznam použitých zdrojů

[1] *Jak se hraje pexeso*. Online. 2020. Dostupné z: https://www.hrackyodjinud.cz/Jak-se-hraje-pexeso-b80351.htm. [cit. 2026-03-27].

[2] MICROSOFT. *Control.Tag Property*. Online. 2026. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.control.tag. [cit. 2026-03-27].

[3] MICROSOFT. *Windows Forms – přehled*. Online. 2025. Dostupné z: https://learn.microsoft.com/cs-cz/dotnet/desktop/winforms/overview/. [cit. 2026-03-27].

[4] MICROSOFT. *Form Třída*. Online. 2026. Dostupné z: https://learn.microsoft.com/cs-cz/dotnet/api/system.windows.forms.form?view=windowsdesktop-8.0. [cit. 2026-03-27].

[5] MICROSOFT. *Control.ControlCollection Class*. Online. 2026. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.control.controlcollection.[cit. 2026-03-27].

[6] MICROSOFT. *Graphics.DrawImage Method*. Online. 2026. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.drawing.graphics.drawimage. [cit. 2026-03-27].

[7] NVIDIA CORPORATION. *Improve Batching Using Texture Atlases*. Online. 2004. Dostupné z: https://download.nvidia.com/developer/NVTextureSuite/Atlas_Tools/Texture_Atlas_Whitepaper.pdf. [cit. 2026-03-27].

[8] MICROSOFT. *How to: Reduce Graphics Flicker with Double Buffering for Forms and Controls*. Online. 2025. Dostupné z: https://learn.microsoft.com/en-us/dotnet/desktop/winforms/advanced/how-to-reduce-graphics-flicker-with-double-buffering-for-forms-and-controls. [cit. 2026-03-27].

[9] MICROSOFT. Control.CreateParams Property. Online. Microsoft.com. 2026. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.control.createparams?view=windowsdesktop-10.0. [cit. 2026-03-27].

[10] MICROSOFT. *9 Variables*. Online. 2025. Dostupné z: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/variables. [cit. 2026-03-27].

[11] MICROSOFT. List<T> Class. Online. 2026. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-10.0. [cit. 2026-03-27].

[12] MICROSOFT. *FileStream Class*. Online. 2025. Dostupné z: https://learn.microsoft.com/en-us/dotnet/api/system.io.filestream?view=net-10.0. [cit. 2026-03-27].

[13] MICROSOFT. *SoundPlayer Třída*. Online. 2026. Dostupné z: https://learn.microsoft.com/cs-cz/dotnet/api/system.media.soundplayer?view=netframework-4.0. [cit. 2026-03-27].

Seznam použitých symbolů a zkratek

UI – User Interface (uživatelské rozhraní)

GUI – Graphic User Interface (grafické uživatelské rozhraní)

I/O – Input/Output (vstupní/výstupní)

.dat – datový soubor

.NET – softwarová platforma

AI – Artificial Inteligence (umělá inteligence)

OOP – Object Oriented Programming (objektově orientované programování)

Seznam obrázků

Obrázek 2.1 Hlavní menu 15
Obrázek 2.2 Metoda pro vytvoření tlačítka reprezentujícího kartičku 17
Obrázek 2.3 Metoda pro získání výřezu kartičky 19
Obrázek 2.4 Rozhraní nastavení 24
Obrázek 2.5 Tabulka skóre 26